Python and AI lessons:

Overview:

Prerequisites:

The roadmap

Lesson 1: Hello World

- Print
- Variables
- f-strings
- Input
- **Exercises:**

Lesson 2: Conditionals

- **The basic shape: if-else**
- elif
- Truthiness
- **Exercises**

Lesson 3: Loops

- **The for loop walks over a collection**
- **range() for counting**
- **while repeats while a condition is true**
- **break and continue**
- **enumerate() gives you the index and the item**
- **zip() walks two collections together**
- **Nested loops**
- **else on a loop (a Python oddity)**
- **Exercises**

Lesson 4: Lists, dicts, and comprehensions

- Lists
- Tuples
- Dicts
- Sets
- Comprehensions
- Exercises

Lesson 5: Functions

- Defining and calling
- Parameters
- Returning multiple values
- Functions are values
- Lambda
- Scope
- Recursion
- Exercises

Lesson 6: Classes

- A basic class
- Printing an object
- Operator overloading
- The reflected methods
- isinstance
- **slots**
- Putting it together: a mini Value
- Exercises

Lesson 7: Autograd, the engine that lets a model learn

- The problem
- The chain rule
- The Value class

- What backward() does
- Worked example
- Using the gradients: gradient descent
- Exercises

Lesson 8, Part 1: Tokens, parameters, and the three helper functions

- 1. The tokenizer: text to numbers
- 2. The parameters: where the knowledge lives
- 3. Three helper functions
- 4. How the output becomes a loss
- Exercises

Lesson 8, Part 2: The `gpt()` function

- Stage 1: Embeddings
- Stage 2: The attention block
- Stage 3: The MLP block
- Stage 4: Scores for the next token
- Putting it together
- Exercises

Lesson 9: Training, Adam, and inference

- The order of the file
- The dataset
- The training loop
- The Adam optimizer
- Inference
- Your finished project
- Exercises

Summary

Where you are now

Overview:

This tutorial is to help an existing PHP or JavaScript programmer to transition to Pyton and create a simple micro AI model. For reference, download micropt.py

Prerequisites:

- This assumes the user is fluent in PHP and it's various frameworks (Laravel, Symphony, etc.) and are familiar with native JavaScript
- The user has installed PyCharm on their local machine
- The user has installed Python on their local machine
- The user has integrated Python with PyCharm

These above four prerequisites will not be covered in this document.

The roadmap

1. **Hello World, variables, input, f-strings** (today)
2. **Conditionals**: `if` / `elif` / `else`, comparisons, `and` / `or` / `not`
3. **Loops**: `for`, `range`, `while`, `break` / `continue`, `enumerate`, `zip`
4. **Lists and dicts**, then comprehensions (the `[x for x in ...]` pattern all over microgpt)
5. **Functions**: parameters, return values, lambdas, scope
6. **Classes**: `self`, `__init__`, and operator overloading
7. **Rebuild the `Value` class** and `backward()`, which is how the model learns
8. **Rebuild the tokenizer, parameters, and `gpt()`** (attention, MLP)
9. **Rebuild the training loop, Adam, and inference**, which gets you to a working microgpt of your own

Lesson 1: Hello World

Print

In PyCharm, right-click your project folder, choose **New** → **Python File**, and name it `lesson1`. Type this in and run it:

python

```
print("Hello, World!")
```

That's the whole program. There's no `<?php` tag, no semicolon, and no `echo`.

Variables

Variables need no declaration:

python

```
language = "Python"year = 2026version = 3.13is_fun = True
```

Python figures out the type itself. The four types here are `str`, `int`, `float`, and `bool`. Note the capital `T` in `True`. Use `type(year)` to see what Python thinks something is.

f-strings

f-strings put values inside text. They're the same idea as JS template literals:

python

```
print(f"I'm learning{language} in{year}")
```

You can also do math inside the braces, and format numbers after a colon:

python

```
print(f"Next year is{year + 1}")print(f"Pi is about{3.14159:.
2f}")   # .2f = two decimal places
```

Input

Input reads what the user types, and it always comes back as a string:

python

```
name = input("What's your name? ")print(f"Nice to meet you,{name}")
```

Because the result is a string, converting it is how you do math with it:

python

```
age = int(input("How old are you? "))print(f"In ten years you'll be{age + 10}")
```

Without `int(...)`, adding 10 would raise an error, since Python won't mix `"5" + 10` the way JavaScript does.

Exercises:

1. Print "Hello, World!" on one line, then your own message on a second line.
2. Ask for the user's name and favorite number, then print a sentence using both. Add 1 to the number in the sentence.
3. Ask for a price (a float) and a quantity (an int), then print the total formatted with two decimal places.
4. Predict what `print(10 / 4)`, `print(10 // 4)`, and `print(10 % 4)` show before you run them. Then check.

Lesson 2: Conditionals

Create a new file called `lesson2`. This is where Python's indentation rule starts to matter.

The basic shape: if-else

```
age = 20

if age >= 18:
    print("Adult")
```

```
else:
    print("Minor")
```

Compared with PHP or JS:

- No parentheses are needed around the condition (they're allowed, just unusual).
- A colon ends the `if` and `else` lines.
- The indented lines are the block, so the indentation is the syntax. Use 4 spaces, which PyCharm inserts when you press Tab. Mixing indentation levels by accident gives an `IndentationError`.

elif

elif replaces `else if` and `elseif`:

```
score = 85

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("Needs work")
```

Python checks from the top and runs only the first branch that matches.

Comparison operators are the ones you know: `==`, `!=`, `<`, `>`, `<=`, `>=`. There's no `===`. Python's `==` doesn't do type juggling, so `5 == "5"` is simply `False`.

Logic operators are words instead of symbols:

```
age = 25
has_ticket = True
```

```python
if age >= 18 and has_ticket:
    print("Come on in")

if not has_ticket or age < 18:
    print("Sorry")
```

Python also lets you chain comparisons the way math does:

```python
if 18 <= age < 65:
    print("Working age")
```

Truthiness

Truthiness. Python treats some values as false in a condition: `0`, `0.0`, `""` (empty string), `[]` (empty list), `{}` (empty dict), and `None`. Everything else counts as true. It's similar to PHP and JS, so this works:

```python
name = input("Name? ")
if name:
    print(f"Hello, {name}")
else:
    print("You didn't type anything")
```

Checking membership with `in` is a handy Python shortcut:

```python
vowels = "aeiou"
letter = "e"

if letter in vowels:
    print("Vowel")
```

It works on strings, lists, and dicts, and `not in` is the opposite. It's what PHP's `in_array` and `str_contains` do, with one readable keyword.

One-line conditional (the ternary equivalent). The condition goes in the middle:

```
status = "Adult" if age >= 18 else "Minor"
```

That's the same as `$status = $age >= 18 ? "Adult" : "Minor";`. You'll see the same pattern in microgpt's `other if isinstance(other, Value) else Value(other)`.

Exercises

1. Ask for a number. Print whether it's positive, negative, or zero.
2. Ask for a temperature in Fahrenheit. Print "Cold" below 50, "Nice" from 50 up to 80, and "Hot" at 80 or above. Use `elif`.
3. Ask for a single letter. Print whether it's a vowel or a consonant, using `in`. Then handle uppercase input (look up `.lower()` and try it).
4. Ask for a year and print whether it's a leap year. The rule is divisible by 4, except years divisible by 100 are not leap years unless they're also divisible by 400. Test it with 1900, 2000, 2024, and 2026.
5. Rewrite exercise 1 or 3 using the one-line conditional, so it's a single `print` with no `if` block.

Exercise 4 is the one to spend time on, because it's where `and`, `or`, and the `%` operator from Lesson 1 come together.

Lesson 3: Loops

Create a new file called `lesson3`. Python has two loop types, `for` and `while`. The `for` loop is the one you'll use most, and it works differently from PHP's and JavaScript's.

The for loop walks over a collection

```
for letter in "hello":
    print(letter)
```

There's no counter, no `$i = 0; $i < 5; $i++`. Python's `for` is closer to PHP's `foreach` or JS's `for...of`. It works on strings, lists, dicts, and anything else you can step through:

```
colors = ["red", "green", "blue"]
for color in colors:
    print(f"I like {color}")
```

range() for counting

When you do want numbers, `range()` produces them:

```
for i in range(5):
    print(i)          # 0, 1, 2, 3, 4
```

Note that it stops *before* the end value, and starts at 0. It takes up to three arguments:

```
range(5)          # 0 to 4
range(2, 6)       # 2 to 5
range(0, 10, 2)   # 0, 2, 4, 6, 8   (start, stop, step)
range(5, 0, -1)   # 5, 4, 3, 2, 1   (counting down)
```

You'll see `for pos_id in range(n):` in microgpt's training loop, and when the loop variable isn't used, the convention is to name it `_`, as in `for _ in range(nin)`.

while repeats while a condition is true

```
count = 3
while count > 0:
    print(count)
    count -= 1
print("Liftoff!")
```

Python has no `++` or `--`, so use `count += 1` or `count -= 1`. If the condition never becomes false you get an infinite loop. If that happens, press the red square (Stop) button in PyCharm's Run panel.

break and continue

`break` exits the loop entirely, and `continue` skips to the next pass:

```
for n in range(1, 11):
    if n % 2 == 0:
        continue      # skip even numbers
    if n > 7:
        break         # stop once we pass 7
    print(n)          # prints 1, 3, 5, 7
```

A common pattern is `while True:` with a `break` inside, which keeps asking until the input is acceptable:

```
while True:
    answer = input("Type 'yes' to continue: ")
    if answer == "yes":
        break
```

enumerate() gives you the index and the item

Instead of tracking a counter yourself:

```
for i, color in enumerate(colors):
    print(i, color)
```

That's the `foreach ($colors as $i => $color)` pattern. Add `enumerate(colors, start=1)` to count from 1.

zip() walks two collections together

You met this in microgpt's `linear()` function:

```
names = ["Ann", "Bob", "Cy"]
scores = [90, 72, 85]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

It stops at the shorter of the two lists.

Nested loops

```
for row in range(3):
    for col in range(3):
        print(row * col, end=" ")
    print()
```

The `end=" "` stops `print` from adding a newline, and the bare `print()` finishes the row. This same trick is why microgpt's progress line overwrites itself, with `end='\r'` returning the cursor to the start of the line.

else on a loop (a Python oddity)

A loop can have an `else` that runs only if the loop finished *without* hitting `break`:

```
for n in range(2, 10):
    if n == 7:
        print("Found 7")
        break
else:
    print("No 7 found")
```

You'll rarely need it, but it's good to recognize when you see it.

Exercises

1. Print the numbers from 1 to 10 with a `for` loop and `range`. Then print them counting down from 10 to 1.
2. Print the multiplication table for a number the user types in (for example, `7 x 1 = 7` through `7 x 10 = 70`).
3. Ask the user for numbers repeatedly with a `while` loop, adding each to a running total. Stop when they type `0`, then print the total.
4. Loop over a string the user types and count how many vowels it contains, using `in` from Lesson 2.

5. Given `names = ["Ann", "Bob", "Cy"]` and `scores = [90, 72, 85]`, use `zip` to print each name and score, and use `enumerate` to number the lines starting at 1.
6. **Challenge:** Print the numbers 1 to 30, but print "Fizz" instead of multiples of 3, "Buzz" instead of multiples of 5, and "FizzBuzz" for multiples of both. This is a classic, and it pulls together Lessons 2 and 3.

Lesson 4: Lists, dicts, and comprehensions

Create a file called `lesson4`. These data structures are all over microgpt: `docs`, `tokens`, `state_dict`, `params`, and `keys` are all lists or dicts.

Lists

A list is an ordered, changeable collection. It's your PHP indexed array or JS array:

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])      # apple
print(fruits[-1])     # cherry (negative counts from the end)
print(len(fruits))    # 3
```

Common operations:

```
fruits.append("date")        # add to the end
fruits.insert(1, "blueberry")# add at index 1
fruits.remove("banana")      # remove by value
last = fruits.pop()          # remove and return the last ite
m
fruits.sort()                # sort in place
print("apple" in fruits)     # True
```

Lists can hold mixed types and other lists. microgpt's weight matrices are lists of lists.

Slicing takes a piece with `[start:stop]`, where stop is not included:

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])    # [20, 30, 40]
```

```
print(nums[:2])     # [10, 20]
print(nums[2:])     # [30, 40, 50]
print(nums[::2])    # [10, 30, 50] (every second item)
print(nums[::-1])   # reversed
```

Strings slice the same way, so `"hello"[1:3]` gives `"el"`.

Copies versus references. This is the gotcha from the PHP comparison:

```
a = [1, 2, 3]
b = a            # same list, two names
b.append(4)
print(a)         # [1, 2, 3, 4], a changed too

c = a.copy()     # a real copy (or a[:])
```

Tuples

A tuple is a list that can't be changed after it's created. It uses parentheses:

```
point = (3, 4)
x, y = point        # unpacking
```

You saw these in the `Value` class, where `children` and `local_grads` are stored as tuples. A one-item tuple needs a trailing comma: `(5,)`.

Dicts

A dict maps keys to values, like a PHP associative array or a JS object:

```
person = {"name": "Ann", "age": 30}
print(person["name"])
person["city"] = "Phoenix"       # add or update
print(person.get("phone", "n/a"))# safe lookup with a default
del person["age"]                # remove
```

Looping gives you several choices:

```
for key in person:                      # keys
    print(key)
for key, value in person.items():       # both
    print(key, value)
```

Using `person["phone"]` for a key that doesn't exist raises a `KeyError`, which is why `.get()` exists. `"name" in person` checks keys.

Sets

A set is an unordered collection of unique items. microgpt uses one to find the unique characters:

```
letters = set("banana")     # {'b', 'a', 'n'}
print(sorted(letters))      # ['a', 'b', 'n']
```

Comprehensions

A comprehension builds a list in a single expression. Here's the loop version:

```
squares = []
for n in range(5):
    squares.append(n * n)
```

And the comprehension version:

```
squares = [n * n for n in range(5)]     # [0, 1, 4, 9, 16]
```

Read it as "n squared, for each n in the range." Add a filter at the end:

```
evens = [n for n in range(10) if n % 2 == 0]
```

You can also transform strings, build dicts, and nest them:

```
upper = [w.upper() for w in ["a", "b"]]
lengths = {w: len(w) for w in ["apple", "fig"]}       # dict
```

```
comprehension
grid = [[0 for _ in range(3)] for _ in range(3)]      # 3x3 g
rid of zeros
```

That last line is the same shape as microgpt's `matrix` lambda.

Useful built-ins to know: `sum()`, `min()`, `max()`, `sorted()`, `len()`, and `list()`.

Exercises

1. Make a list of five numbers. Print the first, the last, the middle three (using a slice), and the list reversed.
2. Start with an empty list and, in a loop, keep asking the user for words until they type `done`. Then print the list sorted and how many words were entered.
3. Given `nums = [4, 8, 15, 16, 23, 42]`, print the sum, the smallest, the largest, and the average, without writing a loop (use the built-in functions).
4. Build a dict of three people and their ages. Loop over it with `.items()` and print each pair. Then add a fourth person and look up someone who isn't there using `.get()` with a default.
5. Write a comprehension that produces the squares of all numbers from 1 to 20 that are divisible by 3.
6. Given `text = "the quick brown fox jumps over the lazy dog"`, build a dict that counts how many times each letter appears (ignoring spaces). Hint: loop over the characters and use `.get(ch, 0) + 1`. Then print the three most common letters (look up `sorted` with a `key=`).
7. **Challenge:** Build a 3×3 multiplication grid as a list of lists using a nested comprehension, then print it one row per line.

Exercise 6 is the closest to microgpt so far, since the tokenizer's job is also mapping characters to numbers. Next up is Lesson 5, functions, which includes `lambda` and how scope works.

Lesson 5: Functions

Create a file called `lesson5`. Functions are where microgpt's structure becomes clear: `linear`, `softmax`, `rmsnorm`, and `gpt` are all plain functions.

Defining and calling

```
def greet(name):
    return f"Hello, {name}"

message = greet("Ann")
print(message)
```

The pieces are `def`, the name, parameters in parentheses, a colon, and an indented body. There's no `function` keyword and no braces. If a function has no `return`, it returns `None`.

Python also lets you document a function with a string right under the `def` line, called a docstring. Ctrl+Q in PyCharm will show it:

```
def area(width, height):
    """Return the area of a rectangle."""
    return width * height
```

Parameters

Default values work as in PHP and JS:

```
def power(base, exponent=2):
    return base ** exponent

print(power(5))        # 25
print(power(2, 10))    # 1024
```

Keyword arguments let you name the parameters when calling, in any order:

```
print(power(exponent=3, base=2))    # 8
```

You've already used these. In `print(..., end=" ")` the `end` is a keyword argument, and so is `key=` in `sorted`.

Variable arguments collect extras:

```python
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3, 4))   # 10
```

Inside the function `numbers` is a tuple. This is like PHP's `...$numbers`. There's also `**kwargs`, which collects named extras into a dict. It's worth recognizing, but you won't need it for microgpt.

The mutable default gotcha. Never use a list or dict as a default value:

```python
def add_item(item, items=[]):     # BAD
    items.append(item)
    return items
```

That default list is created once and shared across every call, so items pile up between calls. The usual fix is:

```python
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

Returning multiple values

A function can return several values at once. It's really returning a tuple, which you unpack:

```python
def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([4, 8, 15, 16])
```

Functions are values

As in JavaScript, you can store functions in variables and pass them around:

```
def shout(text):
    return text.upper()

action = shout
print(action("hi"))        # HI

words = ["pear", "fig", "banana"]
print(sorted(words, key=len))   # sorted by length
```

Passing `key=len` hands the function itself over. There are no parentheses, because you're not calling it yet.

Lambda

A lambda is a tiny one-expression function with no name, like a JS arrow function:

```
double = lambda x: x * 2
print(double(5))     # 10

print(sorted(words, key=lambda w: w[-1]))   # sort by last le
tter
```

Lambdas are best for short throwaway functions like the `key=` above. microgpt uses one for `matrix = lambda nout, nin, std=0.08: ...`, though a normal `def` would do the same job.

Scope

Scope is where Python differs most from PHP.

```
rate = 0.1                      # module level

def with_tax(price):
    return price * (1 + rate)   # reads the outer variable, w
hich is fine
```

Functions can **read** outer variables freely, with no `global` or `use` needed. But **assigning** inside a function creates a new local variable:

```
counter = 0

def bump():
    counter = counter + 1       # UnboundLocalError!
```

To change an outer variable you must declare it:

```
def bump():
    global counter
    counter += 1
```

Modifying a list or dict in place doesn't need `global`, because you're not reassigning the name. That's why `keys[li].append(k)` works inside `gpt()`.

Nested functions are common too. An inner function can read the outer function's variables, and to reassign one it uses `nonlocal`:

```
def make_counter():
    count = 0
    def step():
        nonlocal count
        count += 1
        return count
    return step

c = make_counter()
print(c(), c(), c())    # 1 2 3
```

That's a closure: `step` remembers `count` after `make_counter` has finished. microgpt's `build_topo` inside `backward()` is a nested function that uses `topo` and `visited` from its parent.

Recursion

A function can call itself, and `build_topo` does exactly this to walk the computation graph:

```
def countdown(n):
    if n == 0:
        return
    print(n)
    countdown(n - 1)
```

Every recursive function needs a stopping case (here, `n == 0`).

Exercises

1. Write `average(*numbers)` that accepts any number of arguments and returns their mean. Call it with 3 numbers, then with 6.
2. Write `make_multiplier(n)` that returns a function. For example, `triple = make_multiplier(3)` then `triple(5)` gives 15. This one uses a closure.
3. Use `sorted` with a `lambda` as the key to sort `["pear", "fig", "banana", "kiwi"]` first by length.
4. **Challenge, toward microgpt:** Write `dot(a, b)` that returns the dot product of two equal-length lists, using `zip` and `sum`. For `[1, 2, 3]` and `[4, 5, 6]` you should get 32. This is the heart of microgpt's `linear()` function, so `linear` is really just this applied once per row of a weight matrix.

Lesson 6 is classes, including `self`, `__init__`, and operator overloading, and it's the lesson that makes the `Value` class make sense.

Lesson 6: Classes

Create a file called `lesson6`. This is the lesson that unlocks the `Value` class, the part of microgpt that makes learning possible.

A basic class

```python
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        return f"{self.name} says woof"

rex = Dog("Rex", 3)
print(rex.name)       # Rex
print(rex.bark())     # Rex says woof
```

Compared with PHP:

PHP	Python
`__construct`	`__init__`
`$this`	`self`
`$this->name`	`self.name`
`new Dog("Rex", 3)`	`Dog("Rex", 3)` (no `new`)
declared properties	created by assigning in `__init__`

The big difference is that `self` is written explicitly as the first parameter of every method. You don't pass it when calling. Python supplies it, so `rex.bark()` becomes `Dog.bark(rex)` behind the scenes. Forgetting `self` in the `def` line is the most common beginner error, and it gives a confusing "takes 0 positional arguments but 1 was given" message.

Attributes are created on the fly, and nothing is private by default. A leading underscore, like `_children` in microgpt, is only a convention meaning "internal, please don't touch."

Printing an object

If you print the object, you get an unhelpful result like `<__main__.Dog object at 0x...>`. Define `__repr__` to fix that:

```python
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Dog({self.name!r}, {self.age})"

print(Dog("Rex", 3))    # Dog('Rex', 3)
```

Double-underscore methods like this are called **dunder** (double underscore) or magic methods. Python calls them automatically in response to syntax. `__repr__` is called by `print`, and `__init__` is called when you create the object.

Operator overloading

This is the feature PHP doesn't have. Python calls special methods when you use operators:

You write	Python calls
`a + b`	`a.__add__(b)`
`a - b`	`a.__sub__(b)`
`a * b`	`a.__mul__(b)`
`a / b`	`a.__truediv__(b)`
`a ** b`	`a.__pow__(b)`
`-a`	`a.__neg__()`
`a == b`	`a.__eq__(b)`
`len(a)`	`a.__len__()`

Here's a small example:

```python
class Money:
    def __init__(self, cents):
        self.cents = cents
```

```
    def __add__(self, other):
        return Money(self.cents + other.cents)

    def __repr__(self):
        return f"Money({self.cents})"

print(Money(250) + Money(175))    # Money(425)
```

The `+` created a brand-new `Money` object holding the combined amount, and left both originals alone. microgpt's `Value` does exactly this, except each new object also remembers where it came from.

The reflected methods

What happens with `5 + Money(100)`? Python first tries `int.__add__`, which doesn't know about `Money`, so it falls back to the right-hand object's `__radd__` (reflected add):

```
    def __radd__(self, other):
        return self + other
```

Remember that `sum()` starts with the plain integer `0`, so `sum([a, b, c])` begins as `0 + a`. Without `__radd__` on `Value`, that very first step would fail.

isinstance

```
other = other if isinstance(other, Value) else Value(other)
```

`isinstance(x, SomeClass)` asks whether `x` is that type. This line lets you write `value * 2` as well as `value * other_value`, because a plain number gets wrapped in a `Value` first. It's PHP's `instanceof` as a function.

slots

Normally every object stores its attributes in a dictionary, which costs memory. microgpt creates many thousands of `Value` objects, so it declares:

```
__slots__ = ('data', 'grad', '_children', '_local_grads')
```

That tells Python the exact attribute names up front and skips the dictionary. It's purely an optimization, and the code would behave the same without it.

Putting it together: a mini Value

Here's a stripped-down version of the idea behind microgpt's autograd. Type it out rather than pasting:

```
class Value:
    def __init__(self, data, children=()):
        self.data = data
        self.children = children

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value
(other)
        return Value(self.data + other.data, (self, other))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value
(other)
        return Value(self.data * other.data, (self, other))

    def __repr__(self):
        return f"Value({self.data})"

a = Value(2)
b = Value(3)
c = a * b + 4
print(c)              # Value(10)
print(c.children)     # (Value(6), Value(4))
```

You wrote a normal-looking arithmetic expression, but each step quietly recorded its inputs. Following `children` backward from `c` gives you the whole history of the

calculation. That history is the *computation graph*, and in Lesson 7 we'll use it to work out how much each number contributed to the final result.

Exercises

1. Create a `Rectangle` class with `width` and `height`, and methods `area()` and `perimeter()`. Add a `__repr__` so that `print(rect)` shows `Rectangle(3, 4)`. Make two rectangles and print their areas.
2. Create a `Vector` class holding `x` and `y`. Add `__add__` (component-wise), `__sub__`, and `__mul__` so that `Vector(1, 2) * 3` scales both parts. Add `__repr__` so you can see the results.
3. Type out the mini `Value`. Add `__neg__`, `__sub__`, and `__radd__`, and confirm `sum([Value(1), Value(2), Value(3)])` works.
4. Challenge: Add a `__pow__` method to your mini `Value` so `Value(3) ** 2` gives `Value(9)`. Then compare it with microgpt's version.

Lesson 7: Autograd, the engine that lets a model learn

Create a file called `lesson7`. This is the most important idea in the whole file. Everything else in microgpt just produces a number called the loss. This lesson is how the model finds out which way to adjust each of its 4,192 parameters to make that number smaller.

The problem

Training means nudging every parameter so the loss goes down. To do that you need to know, for each parameter, "if I increase you slightly, does the loss go up or down, and by how much?" That quantity is the **gradient** (a derivative).

A tiny example. Say `L = a * b + c` with `a = 2`, `b = 3`, `c = 4`, so `L = 10`.

- Nudge `a` up by 0.001 and `L` becomes 10.003. Since `a` is multiplied by `b`, the effect is 3 times the nudge, so the gradient of `L` with respect to `a` is **3** (that's `b`).
- Nudge `b` and `L` moves by `a` times the nudge, so the gradient is **2**.

- Nudge `c` and `L` moves one-for-one, so the gradient is **1**.

Each operation has a simple rule for how its output responds to its inputs. These are called **local gradients**:

Operation	Local gradients (output with respect to each input)
`a + b`	`1` and `1`
`a * b`	`b` and `a`
`a ** n`	`n * a**(n-1)`
`log(a)`	`1 / a`
`exp(a)`	`exp(a)`
`relu(a)`	`1` if `a > 0`, else `0`

Each of these local gradients is the third argument you saw passed into `Value(...)` in microgpt.

The chain rule

A real model chains thousands of these operations. The **chain rule** says that to find how the final loss responds to some early value, you multiply the local gradients along the path from the loss back to that value.

In code, each node stores its children and the local gradient for each child. Going backward from the loss, every node hands each child:

```
child.grad += local_grad * node.grad
```

So a node's gradient is its local gradient times the gradient of whatever consumed it, and the loss itself starts with a gradient of 1.

The `+=` matters. If a value is used in more than one place, the contributions from each path add up.

The Value class

Type this out in full. It's the `Value` class from microgpt, plus a `__repr__` so you can print it. I've left out `__slots__`, since it's only a memory optimization.

```
import math

class Value:
    def __init__(self, data, children=(), local_grads=()):
        self.data = data                # the number itself
        self.grad = 0                   # d(loss)/d(this), f
illed in by backward()
        self._children = children       # the Values this on
e was computed from
        self._local_grads = local_grads  # d(this)/d(each chi
ld)

    def __repr__(self):
        return f"Value(data={self.data}, grad={self.grad})"

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value
(other)
        return Value(self.data + other.data, (self, other),
(1, 1))

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value
(other)
        return Value(self.data * other.data, (self, other),
(other.data, self.data))

    def __pow__(self, other):
        return Value(self.data**other, (self,), (other * sel
f.data**(other - 1),))

    def log(self):  return Value(math.log(self.data), (sel
f,), (1 / self.data,))
    def exp(self):  return Value(math.exp(self.data), (sel
f,), (math.exp(self.data),))
```

```
    def relu(self): return Value(max(0, self.data), (self,),
(float(self.data > 0),))

    def __neg__(self):           return self * -1
    def __radd__(self, other):   return self + other
    def __sub__(self, other):    return self + (-other)
    def __rsub__(self, other):   return other + (-self)
    def __rmul__(self, other):   return self * other
    def __truediv__(self, other):  return self * other**-1
    def __rtruediv__(self, other): return other * self**-1

    def backward(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._children:
                    build_topo(child)
                topo.append(v)
        build_topo(self)

        self.grad = 1
        for v in reversed(topo):
            for child, local_grad in zip(v._children, v._loca
l_grads):
                child.grad += local_grad * v.grad
```

Two things to notice:

- Only `+`, , `*`, `log`, `exp`, and `relu` carry real math. Everything else is built from them. Subtraction is adding a negative, negation is multiplying by -1, and division is multiplying by a power of -1. That's why the model needs so few primitives.
- The extra methods such as `__radd__` and `__rmul__` only exist so that plain numbers can sit on either side of an operator.

What backward() does

It has two stages.

1. Order the nodes (topological sort). `build_topo` visits a node's children first and appends the node afterward. So the `topo` list has inputs first and the loss last. It's recursion, like the `countdown` in Lesson 5, with the `visited` set preventing the same node being processed twice.

2. Walk the list in reverse. Reversing puts the loss first, and every node is processed only after everything that uses it has already passed its gradient down. That guarantee is why the order matters. Skip it and a node could pass along an incomplete gradient.

Worked example

Let `L = a * b + a` with `a = 2` and `b = 3`. Notice that `a` is used twice.

```
a = Value(2.0)
b = Value(3.0)
c = a * b          # 6
L = c + a          # 8
L.backward()
print(a.grad, b.grad)    # 4.0 2.0
```

By hand: `L.grad` starts at 1. Then:

1. `L = c + a`, with local grads `(1, 1)`. So `c.grad += 1` and `a.grad += 1`.
2. `c = a * b`, with local grads `(b, a) = (3, 2)`. So `a.grad += 3 * 1` and `b.grad += 2 * 1`.

That gives `a.grad = 1 + 3 = 4` and `b.grad = 2`. The `+=` is what let the two paths to `a` add up. Check it with the maths: `L = ab + a`, so `dL/da = b + 1 = 4` and `dL/db = a = 2`. It matches.

Using the gradients: gradient descent

A gradient says which way the loss increases. To reduce the loss, step the opposite way:

```
x = Value(0.0)
for step in range(50):
    loss = (x - 3) ** 2          # smallest when x is 3
    loss.backward()
    x.data -= 0.1 * x.grad       # step against the gradient
    x.grad = 0                   # reset, because backward() us
es +=
print(x.data)                    # very close to 3
```

That's microgpt's training loop in miniature. Each pass rebuilds the graph during the forward step, calls `backward()`, updates every parameter's `.data`, and then zeroes the gradients. In microgpt the update step is the Adam optimizer, a smarter variant of `x.data -= lr * x.grad`, and the reset is the `p.grad = 0` at the bottom of that loop.

Exercises

1. Type out the `Value` class, then run the worked example. Confirm that `a.grad` is 4.0 and `b.grad` is 2.0.
2. Let `f = (a + b) * (a - b)` with `a = 3` and `b = 2`. Work out the gradients by hand (it's `a² - b²`), then check them with `backward()`. You should get 6 and -4.
3. Run the gradient-descent loop above, then try learning rates of 0.5, 1.0, and 1.1. Describe what happens to `x` each time. (Hint: at 1.0 it bounces, and at 1.1 it blows up. That's why learning rate matters.)
4. **Challenge:** Fit a line. Using the data points `(1, 3)`, `(2, 5)`, `(3, 7)`, start with `w = Value(0.0)` and `b = Value(0.0)`, and loop with learning rate 0.02 for 500 steps: Then call `backward()`, update both `w.data` and `b.data`, and reset both gradients. You should end up near `w = 2` and `b = 1`. This is a model with two parameters, where microgpt has 4,192, and the loop is the same. It also uses `sum()` starting from `0`, so it needs `__radd__`.

   ```
   loss = sum((w * x + b - y) ** 2 for x, y in data)
   ```

Lesson 8 rebuilds the tokenizer, the parameters, and the `gpt()` function itself with attention and the MLP, which is the biggest chunk of microgpt. It could take two sittings.

Lesson 8, Part 1: Tokens, parameters, and the three helper functions

Create a file called `lesson8`. This is the biggest chunk of microgpt, so I'm splitting it. This lesson covers everything the model is *made of*, and Part 2 covers `gpt()`, the function that assembles those pieces. That pushes training and inference to Lesson 9.

1. The tokenizer: text to numbers

A model only works with numbers, so every character has to become an integer ID. microgpt does this in about four lines:

```
docs = ["emma", "olivia", "ava"]

uchars = sorted(set(''.join(docs)))
BOS = len(uchars)
vocab_size = len(uchars) + 1

print(uchars)        # ['a', 'e', 'i', 'l', 'm', 'o', 'v']
print(BOS)           # 7
print(vocab_size)    # 8
```

The pieces, in order:

- `''.join(docs)` glues every name into one long string.
- `set(...)` keeps each distinct character once, and `sorted(...)` puts them in a fixed order.
- A character's **position in `uchars` is its token ID**, so `'a'` is 0 and `'e'` is 1.
- `BOS` ("beginning of sequence") is one extra special token with the next free ID. It doesn't stand for a letter. It marks where a name starts and ends.

To encode a name, wrap it in `BOS` on both sides:

```
doc = "ava"
tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
print(tokens)         # [7, 0, 6, 0, 7]
```

To decode a token ID, look it up: `uchars[0]` gives `'a'`.

In the real file the dataset has 26 letters, so `vocab_size` is 27. That's the number you saw printed on your first run. The training task is "given the tokens so far, predict the next one." For `[7, 0, 6, 0, 7]`, the model sees 7 and should predict 0, then sees 0 and should predict 6, and so on. The last prediction, a `BOS`, teaches it when a name ends.

2. The parameters: where the knowledge lives

Parameters are the numbers the model adjusts during training. They start random and end up holding everything it learned.

```
import random
random.seed(42)

matrix = lambda nout, nin, std=0.08: [[Value(random.gauss(0,
std)) for _ in range(nin)] for _ in range(nout)]
```

That builds a grid with `nout` rows and `nin` columns, each cell a `Value` holding a small random number from a bell curve centered on 0 (`std` is the spread). These are the `Value` objects from Lesson 7, so each one will collect a gradient.

The settings, and the shape of each matrix:

```
n_layer = 1       # how many transformer layers
n_embd = 16       # size of each token's vector
block_size = 16   # longest sequence the model can handle
n_head = 4        # attention heads
head_dim = n_embd // n_head   # 4

state_dict = {
```

```
    'wte':     matrix(vocab_size, n_embd),   # token embeddin
gs:    27 x 16
    'wpe':     matrix(block_size, n_embd),   # position embed
dings: 16 x 16
    'lm_head': matrix(vocab_size, n_embd),   # output layer:
27 x 16
}
for i in range(n_layer):
    state_dict[f'layer{i}.attn_wq'] = matrix(n_embd, n_embd)
# 16 x 16
    state_dict[f'layer{i}.attn_wk'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wv'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.attn_wo'] = matrix(n_embd, n_embd)
    state_dict[f'layer{i}.mlp_fc1'] = matrix(4 * n_embd, n_em
bd)  # 64 x 16
    state_dict[f'layer{i}.mlp_fc2'] = matrix(n_embd, 4 * n_em
bd)  # 16 x 64

params = [p for mat in state_dict.values() for row in mat for
p in row]
print(len(params))   # 4192
```

The last line is a triple-nested comprehension. Read it left to right as three loops: for each matrix, for each row in it, for each value in that row, keep the value. It flattens the whole `state_dict` into one list so the optimizer can loop over every parameter easily.

Where 4,192 comes from. You can verify it by hand:

Matrix	Shape	Count
`wte`	27 × 16	432
`wpe`	16 × 16	256
`lm_head`	27 × 16	432
`attn_wq`, `wk`, `wv`, `wo`	4 of 16 × 16	1,024
`mlp_fc1`	64 × 16	1,024

Matrix	Shape	Count
`mlp_fc2`	16 × 64	1,024
Total		**4,192**

What embeddings are. `wte` is a lookup table with one row per token. Token ID 5 gets row 5, a list of 16 numbers. At first those numbers are random, but training shapes them so that letters used in similar ways end up with similar rows. `wpe` does the same for *position* (first letter, second letter, and so on), since the model needs to know where in the name it is.

3. Three helper functions

`linear` is a matrix times a vector:

```
def linear(x, w):
    return [sum(wi * xi for wi, xi in zip(wo, x)) for wo in
w]
```

`w` is a grid of rows, and `x` is a vector. For each row `wo` it takes the dot product with `x`. That's exactly the `dot` function from the Lesson 5 challenge, run once per row. A 64 × 16 matrix applied to a 16-number vector returns 64 numbers. Nearly all of the model's computation happens here.

`softmax` turns raw scores into probabilities:

```
def softmax(logits):
    max_val = max(val.data for val in logits)
    exps = [(val - max_val).exp() for val in logits]
    total = sum(exps)
    return [e / total for e in exps]
```

The model's raw outputs, called **logits**, can be any numbers, positive or negative. Softmax makes them into a list of positive numbers that add up to 1, where bigger scores get bigger shares. Steps:

1. Find the largest score. Subtracting it from every score changes nothing mathematically, but keeps `exp()` from overflowing on big numbers. Note that

`max_val` is a plain float pulled from `.data`, so it stays out of the computation graph.

2. Apply `exp` to each, which makes everything positive.
3. Divide each by the total so they sum to 1.

`rmsnorm` rescales a vector to a standard size:

```
def rmsnorm(x):
    ms = sum(xi * xi for xi in x) / len(x)
    scale = (ms + 1e-5) ** -0.5
    return [xi * scale for xi in x]
```

It computes the mean of the squares (`ms`), then multiplies every number by `1 / sqrt(ms)`. After that the vector has a root-mean-square size of about 1. Without this, the numbers can drift to huge or tiny values as they pass through layers, which makes training unstable. The `1e-5` just prevents dividing by zero.

All three work on `Value` objects, so every operation inside them extends the computation graph, which is what lets `backward()` reach back through them.

4. How the output becomes a loss

You'll see this in Lesson 9, but it ties softmax to the number you've been watching:

```
zzzprobs = softmax(logits)
loss_t = -probs[target_id].log()
```

The loss for one prediction is the negative log of the probability the model gave the *correct* next token. If the model is confident and right (probability near 1), the loss is near 0. If it's confident and wrong, the loss is large.

At the start, with no learning, all 27 tokens get roughly equal probability, 1/27. The loss is then `-log(1/27)`, which is about **3.30**, the same number I gave you earlier as the "random guessing" benchmark. Your trained run ended at 2.65.

Exercises

1. Build `matrix(3, 4)` using your `Value` class from Lesson 7. Print its number of rows and columns, then flatten it into a list of 12 values with a nested comprehension.
2. Write `count_params(vocab_size, n_embd, block_size, n_layer)` that returns the total from the formula in the table. Confirm 4,192 for the real settings, then see what happens to the count with `n_embd = 32`.
3. Write a plain-float `softmax` using `math.exp` (no `Value`). Check that the output sums to 1, then try `[2.0, 1.0, 0.1]`, and also `[1000, 1001, 1002]` with and without the max-subtraction trick, so you can see what the trick prevents.
4. Compute the loss `-math.log(p)` for `p = 0.9`, `p = 0.5`, `p = 1/27`, and `p = 0.01`. What pattern do you see as the probability of the right answer falls?
5. **Challenge:** Take your `Value` class from Lesson 7 and run `softmax` and the loss on `[Value(2.0), Value(1.0), Value(0.1)]`, treating index 0 as the correct answer. Call `backward()` and print the gradient on each logit. The correct one should be negative and the others positive. What does that tell you about which way each score will move in training?

Lesson 8, Part 2: The `gpt()` function

Create a file called `lesson8_part2`. It needs the `Value` class from Lesson 7 plus everything from Part 1 (the settings, `state_dict`, `linear`, `softmax`, `rmsnorm`), so copy those to the top of the file first.

This part is the transformer itself. The function takes **one token** and its position, and returns a score (a logit) for every possible next token. Here is the whole thing in four stages, and then we'll go through each.

```
def gpt(token_id, pos_id, keys, values):
    # Stage 1: look up and combine embeddings
    tok_emb = state_dict['wte'][token_id]
    pos_emb = state_dict['wpe'][pos_id]
    x = [t + p for t, p in zip(tok_emb, pos_emb)]
    x = rmsnorm(x)

    for li in range(n_layer):
```

```
        # Stage 2: attention block
        ...
        # Stage 3: MLP block
        ...

    # Stage 4: scores for the next token
    logits = linear(x, state_dict['lm_head'])
    return logits
```

Stage 1: Embeddings

```
tok_emb = state_dict['wte'][token_id]     # 16 numbers descri
bing this letter
pos_emb = state_dict['wpe'][pos_id]       # 16 numbers descri
bing this position
x = [t + p for t, p in zip(tok_emb, pos_emb)]
x = rmsnorm(x)
```

The row of `wte` for the token and the row of `wpe` for the position are added element by element. The result `x` is a list of 16 numbers meaning "this letter, at this spot." From here on, `x` is the model's working vector, and it stays 16 numbers long all the way through. The `rmsnorm` rescales it to a standard size.

Stage 2: The attention block

Attention is how a token gets information from the earlier tokens. When predicting what follows the "m" in "emma," the model needs to know about the "e" that came before. Here is the code:

```
x_residual = x
x = rmsnorm(x)
q = linear(x, state_dict[f'layer{li}.attn_wq'])
k = linear(x, state_dict[f'layer{li}.attn_wk'])
v = linear(x, state_dict[f'layer{li}.attn_wv'])
```

```
keys[li].append(k)
values[li].append(v)
```

Query, key, value. The current vector is turned into three new 16-number vectors by three different weight matrices:

- **q (query):** "What am I looking for?"
- **k (key):** "What do I contain, for anyone searching?"
- **v (value):** "What information do I hand over if someone picks me?"

A good analogy is a library. Your query is compared against every book's label (the keys), and the books that match best contribute the most of their contents (the values).

The cache. `keys[li]` and `values[li]` are lists that grow by one entry per token. Because the model is called once per position, these lists remember the k and v of every earlier token in the name. By the time you reach the third letter, `keys[0]` holds three entries. The lists are created fresh at the start of each name, which is why `gpt()` takes them as arguments. This also means a token can only see itself and **earlier** tokens, never later ones, since later ones haven't been appended yet. That's the rule a language model must follow, because at prediction time the future doesn't exist.

Then it does the attention work separately for each of the 4 heads:

```
x_attn = []
for h in range(n_head):
    hs = h * head_dim
    q_h = q[hs:hs+head_dim]
    k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
    v_h = [vi[hs:hs+head_dim] for vi in values[li]]
    attn_logits = [sum(q_h[j] * k_h[t][j] for j in range(head
_dim)) / head_dim**0.5
                   for t in range(len(k_h))]
    attn_weights = softmax(attn_logits)
    head_out = [sum(attn_weights[t] * v_h[t][j] for t in rang
e(len(v_h)))
```

```
                    for j in range(head_dim)]
        x_attn.extend(head_out)
```

Heads. The 16 numbers are split into 4 slices of 4 (`head_dim`). Each head works only on its own slice, so it can learn to look for something different. Slicing is the `[hs:hs+head_dim]` from Lesson 4.

Scores. `attn_logits` has one number per earlier token (including the current one): the dot product of this token's query with that token's key. A large dot product means a strong match. The division by `head_dim**0.5` (that's 2) keeps the scores from growing too large as vectors get longer, which would make the softmax too extreme.

Weights. `softmax` turns those scores into percentages that add up to 1. For the third letter, this might be something like 10% on the first letter, 20% on the second, and 70% on itself.

Mixing. `head_out` is the weighted average of the **value** vectors, where each token contributes according to its weight. So if a head decides the previous letter matters most, its output looks mostly like that letter's value vector.

Joining the heads. `x_attn.extend(head_out)` appends each head's 4 numbers to the list, so after the loop `x_attn` is back to 16 numbers. (`extend` adds the items of a list, where `append` would add the list as one item.)

Then the result is projected once more and added to the original:

```
x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
x = [a + b for a, b in zip(x, x_residual)]
```

The residual connection. That last line adds the vector from *before* the attention block back in. The block only has to learn a *correction* to `x`, and it gives gradients a direct path backward through the network, which makes training much more stable. It's the reason `x_residual = x` was saved at the top of the block. That's safe because `x` is reassigned to new lists afterward and the old list is never modified, the reference behavior from Lesson 4.

Stage 3: The MLP block

```
x_residual = x
x = rmsnorm(x)
x = linear(x, state_dict[f'layer{li}.mlp_fc1'])    # 16 -> 64
x = [xi.relu() for xi in x]
x = linear(x, state_dict[f'layer{li}.mlp_fc2'])    # 64 -> 16
x = [a + b for a, b in zip(x, x_residual)]
```

Where attention moves information *between* tokens, the MLP (multi-layer perceptron) processes each token on its own. It expands the 16 numbers to 64, applies `relu` (which zeroes negatives, giving the network the nonlinearity it needs to learn anything beyond straight-line relationships), and then shrinks back to 16. The widening gives the model more room to compute with. Then the same residual trick adds the original back in.

Stage 4: Scores for the next token

```
logits = linear(x, state_dict['lm_head'])    # 16 -> 27
```

One last linear step turns the 16-number vector into 27 scores, one per token in the vocabulary. Run them through `softmax` and you have the model's probability for each possible next letter, which feeds the loss calculation from Part 1.

Putting it together

The whole function, as it appears in microgpt:

```
def gpt(token_id, pos_id, keys, values):
    tok_emb = state_dict['wte'][token_id]
    pos_emb = state_dict['wpe'][pos_id]
    x = [t + p for t, p in zip(tok_emb, pos_emb)]
    x = rmsnorm(x)

    for li in range(n_layer):
        # 1) Multi-head Attention block
        x_residual = x
        x = rmsnorm(x)
```

```
        q = linear(x, state_dict[f'layer{li}.attn_wq'])
        k = linear(x, state_dict[f'layer{li}.attn_wk'])
        v = linear(x, state_dict[f'layer{li}.attn_wv'])
        keys[li].append(k)
        values[li].append(v)
        x_attn = []
        for h in range(n_head):
            hs = h * head_dim
            q_h = q[hs:hs+head_dim]
            k_h = [ki[hs:hs+head_dim] for ki in keys[li]]
            v_h = [vi[hs:hs+head_dim] for vi in values[li]]
            attn_logits = [sum(q_h[j] * k_h[t][j] for j in ra
nge(head_dim)) / head_dim**0.5
                           for t in range(len(k_h))]
            attn_weights = softmax(attn_logits)
            head_out = [sum(attn_weights[t] * v_h[t][j] for t
in range(len(v_h)))
                        for j in range(head_dim)]
            x_attn.extend(head_out)
        x = linear(x_attn, state_dict[f'layer{li}.attn_wo'])
        x = [a + b for a, b in zip(x, x_residual)]
        # 2) MLP block
        x_residual = x
        x = rmsnorm(x)
        x = linear(x, state_dict[f'layer{li}.mlp_fc1'])
        x = [xi.relu() for xi in x]
        x = linear(x, state_dict[f'layer{li}.mlp_fc2'])
        x = [a + b for a, b in zip(x, x_residual)]

    logits = linear(x, state_dict['lm_head'])
    return logits
```

If you compare it to GPT-2, there are three small simplifications: `rmsnorm` instead of layer norm, no bias terms, and ReLU instead of GeLU. Otherwise it is the same architecture that large models use, just scaled down.

Exercises

1. In your file, call `gpt(BOS, 0, keys, values)` with `keys, values = [[] for _ in range(n_layer)], [[] for _ in range(n_layer)]`. Print `len(logits)` and confirm it's 27.
2. Feed in the tokens of `"ava"` one at a time, reusing the same `keys` and `values`, and print `len(keys[0])` after each call. It should go 1, 2, 3, 4. Then explain why this is the "memory" of the model.
3. Compute the loss for the first prediction (`BOS` predicting `'a'`) with `-softmax(logits)[target].log()`. It should be near 3.3.
4. **Challenge:** Write a plain-float `attention(q, keys, values)` for a single head with 3 stored tokens using small made-up vectors, and verify by hand that the weights sum to 1 and the output is a weighted average of the values.

Exercises 2, 4, and 5 are the essential ones, because they check that your copy of `gpt()` works and the plumbing is right. In Lesson 9 we'll finish with the training loop, the Adam optimizer, and inference, which gets you to a running microgpt of your own. Paste any errors you hit and I'll help sort them out.

Lesson 9: Training, Adam, and inference

Create a file called `lesson9`. This lesson finishes microgpt, so it needs everything from Lessons 7 and 8 above it. By the end you'll have a complete, working copy of your own.

The order of the file

This is the order everything appears in microgpt, so you can assemble it:

1. Imports and `random.seed(42)`
2. Load the dataset into `docs` and shuffle it
3. Tokenizer (`uchars`, `BOS`, `vocab_size`)
4. The `Value` class (Lesson 7)
5. Settings, `state_dict`, and `params` (Lesson 8, Part 1)
6. `linear`, `softmax`, `rmsnorm` (Lesson 8, Part 1)
7. `gpt()` (Lesson 8, Part 2)

8. Optimizer setup, training loop, and inference (today)

Python reads top to bottom with no hoisting, so each piece must sit above the code that uses it.

The dataset

```
import os
import random
random.seed(42)

if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/m
akemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.st
rip()]
random.shuffle(docs)
print(f"num docs: {len(docs)}")
```

It downloads the names file once if it isn't already there, reads it into a list with a comprehension that skips blank lines, and shuffles it. The seed makes the shuffle and the random initial weights the same on every run, so your output is reproducible and can be compared with mine or with the original.

The training loop

One pass through the loop is one **step**, and each step trains on a single name:

```
num_steps = 1000
for step in range(num_steps):

    doc = docs[step % len(docs)]
    tokens = [BOS] + [uchars.index(ch) for ch in doc] + [BOS]
    n = min(block_size, len(tokens) - 1)
```

`step % len(docs)` cycles through the dataset (the `%` from Lesson 1). With 1,000 steps and 32,033 names, the model only ever sees the first 1,000 names of the shuffled list, which is a very small slice. `n` is how many predictions to make, capped at `block_size` because the position table `wpe` only has 16 rows.

The forward pass:

```
    keys, values = [[] for _ in range(n_layer)], [[] for _ in
range(n_layer)]
    losses = []
    for pos_id in range(n):
        token_id, target_id = tokens[pos_id], tokens[pos_id +
1]
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax(logits)
        loss_t = -probs[target_id].log()
        losses.append(loss_t)
    loss = (1 / n) * sum(losses)
```

Each pass starts with fresh, empty `keys` and `values` so the model begins every name with no memory. Then for each position it feeds in the current token, gets scores for the next one, turns them into probabilities, and records `-log(probability of the right answer)`. The name's loss is the average over its positions. Every operation along the way built part of the computation graph from Lesson 7, so by the end `loss` is the root of one very large graph.

The backward pass:

```
    loss.backward()
```

One call, and every parameter's `.grad` is filled in. This is the `backward()` you built in Lesson 7, now working over thousands of nodes.

The Adam optimizer

In Lesson 7 you updated a parameter with `x.data -= lr * x.grad`. That's plain gradient descent, and it works, but it's touchy. Adam is the refinement almost every modern model uses.

Setup, above the loop:

```
learning_rate, beta1, beta2, eps_adam = 0.01, 0.85, 0.99, 1e-8
m = [0.0] * len(params)    # one slot per parameter: running average of gradients
v = [0.0] * len(params)    # one slot per parameter: running average of squared gradients
```

`[0.0] * len(params)` builds a list of 4,192 zeros. Then, inside the loop after `backward()`:

```
    lr_t = learning_rate * (1 - step / num_steps)
    for i, p in enumerate(params):
        m[i] = beta1 * m[i] + (1 - beta1) * p.grad
        v[i] = beta2 * v[i] + (1 - beta2) * p.grad ** 2
        m_hat = m[i] / (1 - beta1 ** (step + 1))
        v_hat = v[i] / (1 - beta2 ** (step + 1))
        p.data -= lr_t * m_hat / (v_hat ** 0.5 + eps_adam)
        p.grad = 0
```

What each piece does:

- `m` **is momentum.** It's a smoothed average of recent gradients, so one noisy step doesn't yank a parameter around. Since each step sees only one name, the gradients are very noisy, and this smoothing helps a lot.
- `v` **tracks gradient size.** It's a running average of the *squared* gradients. Dividing by its square root means parameters with consistently huge gradients take proportionally smaller steps, and parameters with tiny gradients take proportionally larger ones. Every parameter ends up moving at a similar pace.
- `m_hat` **and** `v_hat` **are bias corrections.** Both averages start at zero, so early on they're pulled too low. Dividing by `1 - beta ** (step + 1)` fixes that, and the correction fades to nothing as steps accumulate.
- `eps_adam` (`1e-8`) prevents dividing by zero.

- `lr_t` **decays linearly** from 0.01 at the start to 0 at the end, so the model makes big moves early and fine adjustments late.
- `p.grad = 0` resets the gradient, because `backward()` accumulates with `+=`. Forget it and the gradients from every step would pile up, as you saw with the gradient-descent loop in Lesson 7.

The same `enumerate` idiom from Lesson 3 gives the index `i` for the `m` and `v` lists as well as the parameter.

The progress line:

```
    print(f"step {step+1:4d} / {num_steps:4d} | loss {loss.da
ta:.4f}", end='\r')
```

`:4d` pads the number to 4 characters, `:.4f` shows four decimals, and `end='\r'` returns the cursor to the line start so the line overwrites itself instead of scrolling. That's why you saw only one line on your first run.

Because each loss is from a single name, it bounces around from step to step. A short name can come out much lower or higher than a long one. The trend is what matters, not any single number.

Inference

Once training finishes, the weights are frozen and the model can generate:

```
temperature = 0.5
print("\n--- inference (new, hallucinated names) ---")
for sample_idx in range(20):
    keys, values = [[] for _ in range(n_layer)], [[] for _ in
range(n_layer)]
    token_id = BOS
    sample = []
    for pos_id in range(block_size):
        logits = gpt(token_id, pos_id, keys, values)
        probs = softmax([l / temperature for l in logits])
        token_id = random.choices(range(vocab_size), weights=
[p.data for p in probs])[0]
```

```
        if token_id == BOS:
            break
        sample.append(uchars[token_id])
    print(f"sample {sample_idx+1:2d}: {''.join(sample)}")
```

Here's the process for one name:

1. Start with the `BOS` token, meaning "a new name begins."
2. Ask `gpt()` for the scores for the next token.
3. Divide the scores by `temperature` and apply softmax.
4. Pick the next token at random, weighted by those probabilities (`random.choices`, which returns a list, so `[0]` takes the one item).
5. If the token is `BOS`, the model is saying "the name ends here," so stop. Otherwise look up its letter, add it to `sample`, and **feed the new token back in as the next input.**

That last step is how ChatGPT-style models write too: predict one token, append it, and predict the next from everything so far.

Temperature controls how adventurous the picks are. Dividing the scores by a number below 1 exaggerates the gaps, so the most likely letters dominate and the names are safer and more repetitive. A value near 1 keeps the model's raw probabilities, so you get more variety and more oddities. Rolling dice with `random.choices` rather than always taking the top letter is why you get different names every time.

Note that inference never calls `backward()` or updates anything, and no `p.grad` is touched. The graph still gets built, which is wasteful but keeps the code simple.

Your finished project

The best final exercise is to type the whole file out from a blank page, in the order listed above, then run it. When it matches your first run's output (the same seed gives the same `num docs`, `vocab size`, `num params`, and first loss), you've rebuilt it correctly. If you compare your file to the original and a number differs, hunt for the difference. Often it's a one-character typo, and finding it is excellent practice.

Exercises

1. Run the finished file and confirm it prints `num docs: 32033`, `vocab size: 27`, and `num params: 4192`.
2. Print the loss at step 1 instead of only at the end. It should be near 3.3, the random-guessing level. Explain why.
3. Try `temperature` of 0.1, 0.5, 1.0, and 1.5 with the same trained model (the cleanest way is to put the inference code in a function that takes `temperature`). Describe how the names change. What goes wrong at the extremes?
4. Replace Adam with plain gradient descent (`p.data -= learning_rate * p.grad`). Try learning rates of 0.1, 0.01, and 0.001 and compare. This shows what Adam buys you.
5. **Challenge:** Train on a different dataset. Replace `input.txt` with a text file of your own, one item per line (cities, band names, ham radio call-sign-style words, or film titles). Check whether `block_size` is long enough for your longest line. Does the model pick up the style?

Summary

As a PHP developer, you are familiar with PhpUnit for unit testing as well as the three A's (Arrange, Act, Asset) when creating a unit test. When creating a model, thinkit as a unit test that fixes itself: **Arrange**, **Act**, Assert, **Adjust**, **Answer.**

Step	In microgpt
Arrange	Load the dataset, build the tokenizer, create the parameters (`state_dict`, `params`), set up Adam's buffers
Act	Run the forward pass: `gpt()` turns tokens into scores, and `softmax` turns scores into probabilities
Assert	Compute the loss: `-log(probability of the correct next letter)`
Adjust	`loss.backward()` works out each parameter's gradient, then Adam nudges every parameter and resets the gradients to 0
Answer	The inference loop, where the frozen model generates names

In a unit test, you run Arrange, Act, Assert one time and you're done. A short way to think of models is to say "**Arrange once, then Act, Assert, Adjust on repeat, and finally Answer once**."

Where you are now

Take a moment to look at what you've done. You started with `print("Hello, World!")` and finished by rebuilding a complete GPT: a tokenizer, an autograd engine, a transformer with attention, an optimizer, and a text generator, in plain Python with no libraries. The 4,192-parameter model here and the models behind ChatGPT share the same algorithm. Karpathy's line from the file is accurate: "This file is the complete algorithm. Everything else is just efficiency."

Here's what that "efficiency" covers and where to go next:

- **Vectorized math.** Real libraries like PyTorch and NumPy do thousands of these multiply-adds at once on a GPU, where this file does them one scalar at a time. That's why your run took minutes.
- **Scale.** More layers, wider vectors, longer context, billions of parameters, and far more data.
- **Better tokens.** Real models split text into word pieces rather than single letters.

If you want to continue, the natural next steps are the repos:

- **nanoGPT** is the same model rewritten in PyTorch, so you'll recognize every part.
- **makemore** builds up to the same idea more gradually.
- Karpathy's video lectures to go with them are called "Neural Networks: Zero to Hero."
- And the Raschka repo goes further toward a full-scale model.